



The life of a scene

Overview

Chapters 1-3 used scenes as boxes to put code in. This chapter opens the box: the exact order of a scene's methods and events, what happens in a frame, and the scene manager's other powers - `launch`, `pause` / `resume`, `sleep` / `wake`, `stop`, `remove`. The Phaser ideas are the **lifecycle**, **scene events**, **parallel scenes** (a HUD and a pause menu) and **cleaning up listeners**; the OO ideas are **overriding a method and calling** `super` (a `Sprite`'s `preUpdate`) and the **context** argument of a listener (JavaScript's `this` compared with Java's method references).

The chapter has one big practical lesson that students will need in every later chapter: **Phaser clears up a scene's game objects, timers, tweens and input listeners when it shuts down, but not listeners on `this.events` or on longer-lived emitters.** Leaks do not crash at once; they make things happen twice, three times... and then crash. The logger's leak mode makes this visible.

Prerequisites

- Chapter 1: `preUpdate` on an `Image`, `delta`, `override`
- Chapter 2: `scene.start(key, data)`, `init(data)`, scenes are reused, `on` and `once`, the registry
- Chapter 3: the loader and `preload()` (only lightly needed here)
- Java: overriding and `super.method()`; listeners and method references (`this::onClick`)

Learning outcomes

Students can:

1. state the order in which Phaser calls a scene's constructor, `init`, `preload`, `create` and `update`, which of them run once and which on every start, and where game objects'

- `preUpdate` comes in a frame
- 2. listen for scene events (`CREATE` , `PAUSE` , `RESUME` , `SLEEP` , `WAKE` , `SHUTDOWN` , `DESTROY` , `UPDATE` , ...) with a method and a context, and remove the listener again
- 3. override `preUpdate` in a `Sprite` subclass correctly (`protected override` , calling `super`)
- 4. run scenes side by side with `launch` , and choose between `pause` , `sleep` and `stop`
- 5. explain which listeners Phaser removes when a scene shuts down, and clean up the rest in `SHUTDOWN`
- 6. measure time in a pausable game with timers or `delta` , not `time / this.time.now`
- 7. build a HUD scene and a pause menu scene around a game scene

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Recap Chapter 2's trap (fields keep old values). "Today: exactly what Phaser calls, and when"
0:05 - 0:20	Slides 3-5: the lifecycle diagram, then live : run the logger, read the log from the top. Restart with 5 and ask "what is missing?" (the constructor)
0:20 - 0:30	Slides 6-7: one frame in order; scene events; the context argument (Java <code>this::onClick</code> comparison)
0:30 - 0:35	Slide 8: <code>Sprite.preUpdate</code> - live : press S in the logger, then delete <code>super.preUpdate</code> in <code>Spinner.ts</code> and rebuild
0:35 - 0:45	Slides 9-10: the scene manager table; pause vs sleep (press 1, 2, 3, 4); requests are queued
0:45 - 0:55	Slides 11-12: cleaning up. Live : L then 5, 5 - watch every line of the log appear three times. Ask for the fix before showing <code>onShutdown()</code>
0:55 - 1:00	Slide 13: <code>time</code> vs <code>delta</code> - the question "what does <code>time</code> do while paused?"

Lab (2 hours)

Time	Activity
0:00 - 0:20	Everyone runs the logger and fills in a table: for pause, sleep, stop and remove - is it updated? drawn? do its game objects survive? what events? (Answers: the chapter's table)
0:20 - 0:35	Challenge 1
0:35 - 0:55	Play <code>ch04_pause_and_hud</code> ; slides 14-18. Read <code>GameScene</code> , <code>HudScene</code> , <code>PauseScene</code> together; find the three places where something is cleaned up in <code>SHUTDOWN</code> or by <code>stop</code>
0:55 - 1:40	Challenges 2-4
1:40 - 2:00	Challenges 5-6 for the fast; the rest finish 3-4. Collect the "which camera fades?" answers from challenge 5

Key points to stress

- **The constructor runs once.** Everything that must happen on every start goes in `init()` or `create()` . Everything that must happen once for the whole game (animations, registry defaults) must survive `create()` running again - check with `exists` , or do it in a scene that only runs once
- **Scene events are on `this.events` , and that emitter is reused.** This is the surprise: students assume shutting a scene down clears everything. It clears game objects, timers, tweens and input listeners - not listeners on `this.events` , `this.game.events` , `this.registry.events` or other scenes' emitters
- **Paused scenes are drawn; sleeping scenes are not; neither is updated nor hears input.** So the scene that reads the "unpause" key must be a different, running scene
- `start` **stops the caller**; `launch` **does not**. The most common HUD bug is `this.scene.start(HUD)` from the game scene
- `restart()` **has no key**. It restarts its own scene only
- **Scene manager calls are queued** (except `add` and `remove`). Code after `this.scene.start(...)` still runs, in the old scene
- `time` **is the game's clock and keeps going during a pause**. Timers and `delta` are the scene's, and stop. `this.time.now` is only brought up to date while the scene steps

Common problems and errors

What students see	Cause	Fix
This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	preUpdate in a Sprite subclass without override	add override (a Chapter 1 Image did not need it - a Sprite has its own preUpdate
Class 'Spinner' incorrectly extends base class 'Sprite'. / Property 'preUpdate' is private in type 'Spinner' but not in type 'Sprite'.	private override preUpdate	protected (or public) - a subclass cannot make a method less visible
A sprite moves but its animation is stuck on one frame; no error	super.preUpdate(time, delta) missing	call it, first thing in preUpdate
Console: AnimationManager key already exists: spin after a restart	this.anims.create(...) in create() , run again on restart	if (!this.anims.exists(key)) , or create animations in a scene that runs once
After each restart, events happen twice, three times... (sounds doubled, score counted twice)	listeners added in create() to this.events , this.game.events , this.registry.events or another scene's events , never removed	remove them in a SHUTDOWN handler: off(event, method, this)
off(...) does not remove the listener	on and off were each given a new arrow function	pass the same method and context to both, or keep the function in a variable

What students see	Cause	Fix
The pause menu opens but the game can never be unpaused	the unpause key is read by the paused scene, which hears no input	read it in the pause scene (which is running)
The game disappears when the HUD or pause menu appears	<code>this.scene.start(HUD_SCENE)</code> - <code>start</code> stops the calling scene	<code>this.scene.launch(...)</code>
Argument of type 'string' is not assignable to parameter of type 'object'.	<code>this.scene.restart(GAME_SCENE)</code> - <code>restart</code> takes data, not a key	<code>this.scene.start(GAME_SCENE)</code> from the other scene, or <code>restart()</code> inside the scene itse
Property 'score' does not exist on type 'Scene'.	<code>this.scene.get(GAME_SCENE).score</code> - <code>get</code> returns a plain <code>Scene</code> type	better: send the value as event data or registry value. If it really must be read, <code>this.scene.get(GAME_SCENE)</code> as <code>GameScene</code> and a public gette
TypeError: Cannot read properties of null (reading 'queue') in the console when removing a scene from a key press	<code>this.scene.remove(...)</code> runs immediately, in the middle of Phaser dispatching the key to every scene	defer it: <code>this.time.delayedCall(0, () => this.scene.remove(key))</code>
The HUD shows nothing until the first change	it listened for events the game emitted in its <code>create()</code> , before the HUD had started (launch is queued)	pass the starting values as <code>launch</code> data
The invulnerable time / power-up ends early if the game was paused	measured with <code>this.time.now</code> or <code>time</code>	use <code>this.time.delayedCall</code> , or add up <code>delta</code>

Discussion questions

- `DemoScene` 's constructor cannot use `this.log()` , and `EventLog` is a plain class that any file can import. What is available to a scene in its constructor, and why? (Only what `super(...)` sets up; Phaser gives it its systems when the game boots it.)
- The HUD listens to `GameScene` 's events rather than `GameScene` calling HUD methods. What would break, or become harder, if `GameScene` called `hud.showScore(...)` directly? (The HUD might not have started yet; `GameScene` would depend on the HUD existing; a second HUD is harder.)
- When should a scene be paused, slept, or stopped? Give a game example of each
- Why does the leak in leak mode make the log *triple* after two restarts, not double? Why do the leaked `UPDATE` listeners appear *before* `Spinner.preUpdate()` in the log? (Phaser's own update list re-registers its listener on each start, after the leftovers.)
- In challenge 5, why can the pause scene's camera fade the whole screen, but the game scene's cannot?

Extension ideas

- Add a `TRANSITION` between two scenes with `this.scene.transition({ target, duration, moveAbove })` and log its events (`transitionout` , `transitionstart` , `transitioncomplete`)
- Make the logger's frame limit adjustable while running: `this.game.loop.setFPSLimit(n)` (Phaser 4.2) on the number keys, and watch the coin's steps get bigger
- `this.scene.switch(key)` sleeps the current scene and starts (or wakes) another: build a two-room game where each room keeps its state when you leave it
- Run two copies of the same scene class side by side with `this.scene.add(key, new GameScene(key))` and camera viewports - a split screen

Assessment ideas

- Written: give a sequence of calls (`launch` , `pause` , `sleep` , `wake` , `stop` , `start`) and ask for the log a lifecycle logger would print, including which lifecycle methods run again

- Code reading: a scene with `this.game.events.on("blur", () => this.pauseGame())` in `create()` and no clean-up. Ask what happens after three restarts, how to prove it, and for the fix
- Practical: "add a game-over scene launched over the paused game, showing the score, with R to play again" - checks `launch` , `pause` , data, and restarting a paused scene

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Count the frames drawn

Project: `solutions/ch04_challenge_1_count_the_frames` (from `ch04_lifecycle_logger`)

Goal: listen for a scene event; see for yourself that a paused scene is drawn and a sleeping one is not; and notice which scene can report on which.

`src/scenes/DemoScene.ts`

```
private onRender(): void {  
    // CHALLENGE 1: RENDER happens once each time the scene is drawn - even  
    while it is paused  
    this.renderers = this.renderers + 1;  
}
```

```
public getUpdates(): number {  
    return this.updates;  
}  
  
public getRenderers(): number {  
    return this.renderers;  
}
```

`src/scenes/LogScene.ts`

```
// CHALLENGE 1: ask DemoScene for its counts  
counts = `      updates: ${demo.getUpdates()}    renders:  
${demo.getRenderers()}`;
```

plus `this.renderers = 0;` in `init()`, so a restart starts the count again.

Why not in `DemoScene` 's own text? Its text is changed in `update()` - and `update()` is exactly what stops when the scene is paused. The count would freeze on screen even though it is still going up. `LogScene` is always running, so it can show it.

Tested with the harness: after 1 second paused, updates stayed at 32 while renders went from 32 to 76; after putting the scene to sleep, both stopped (35 and 89, unchanged a second later).

Look for: the counter reset in `init()` ; getters rather than making the fields public (both work - the getters say "read only"); a correct answer to "which numbers stop" - paused: updates; asleep: both. Students who put the count in `update()` (`renders++` there) have counted updates twice, not renders - a good discussion point.

2. Thirty frames a second

Project: `solutions/ch04_challenge_2_thirty_fps` (from `ch04_pause_and_hud`)

Goal: the `fps` config, and understanding what `actualFps` does and does not measure.

`src/main.ts`

```
fps: {  
  target: 60,  
  limit: 30,  
},
```

`src/scenes/HudScene.ts`

```

override update(_time: number, delta: number): void {
    this.framesCounted = this.framesCounted + 1;
    this.timeCounted = this.timeCounted + delta;

    if (this.timeCounted >= 1000) {
        const fps = this.framesCounted * 1000 / this.timeCounted;
        this.fpsText.setText(`fps: ${fps.toFixed(1)} (browser:
        ${this.game.loop.actualFps.toFixed(1)})`);
        this.framesCounted = 0;
        this.timeCounted = 0;
    }
}

```

With a limit, Phaser still receives every browser frame; it just skips calling the game step until enough time has built up. `actualFps` counts every browser frame, so it stays near 60. The HUD's `update()` runs once per real game step, so counting its calls gives the true rate. The HUD is a good home for it because it keeps running while the game is paused.

Tested with the harness (the headless browser offers about 48 frames a second): the HUD showed `fps: 26.5 (browser: 48.6)`. On a 60 Hz screen it shows 30. Holding the right arrow for a second moved the player about the same distance with and without the limit.

Look for: the counter based on `delta`, or on `time` differences - both fine; resetting the counters in `init()`. Many students will first display `actualFps`, see 60, and decide the limit "does not work" - worth letting them find that, then asking what `actualFps` is counting.

3. How long was I away?

Project: `solutions/ch04_challenge_3_time_away` (from `ch04_pause_and_hud`)

Goal: a scene that runs while another is paused; passing data with `resume`; a listener on `this.events` that must be removed.

`src/scenes/PauseScene.ts`

```

// CHALLENGE 3: this scene is running while the game is paused, so its
update() does the counting
override update(_time: number, delta: number): void {
    this.pausedFor = this.pausedFor + delta;
    this.awayText.setText(`Paused for ${this.pausedFor / 1000}.toFixed(1)}
s`);
}

// resume the game, and take this scene away
private carryOn(): void {
    // CHALLENGE 3: resume() can hand data to the scene it resumes - it
    arrives with the RESUME event
    const data: ResumeData = { pausedFor: this.pausedFor };
    this.scene.resume(GAME_SCENE, data);
    this.scene.stop();
}

```

src/scenes/GameScene.ts

```

// CHALLENGE 3: hear about each resume, and the data PauseScene sends with
it
this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);

```

```

// CHALLENGE 3: listeners on this.events outlive a shutdown - without this,
every restart
// would add another, and a pause would be counted twice, three times, ...
this.events.off(Phaser.Scenes.Events.RESUME, this.onResume, this);

```

```

// CHALLENGE 3: the RESUME event's listener is given the scene's Systems,
then the data
private onResume(_sys: Phaser.Scenes.Systems, data: ResumeData): void {
    this.timePaused = this.timePaused + data.pausedFor;
    this.events.emit(PAUSED_CHANGED, this.timePaused / 1000);
}

```

and the HUD listens for `PAUSED_CHANGED` like the other three events (and removes that listener in its `SHUTDOWN` handler too).

The "counted once" requirement is the point of the challenge: the `RESUME` listener is on `GameScene`'s own `this.events`, which Phaser does not clear on shutdown. Tested with

the harness: two restarts, then a pause - the HUD showed one pause's worth of time, and `listenerCount("resume")` was 1.

Look for: `pausedFor` reset in `init()` (the pause scene object is reused for every pause); the `off` in `SHUTDOWN`. Other good answers: the pause scene writes the total into the registry, or emits an event on `GameScene`'s emitter. A tempting wrong answer: read `this.time.now` in `GameScene` on `PAUSE` and on `RESUME`, and subtract. It always gives 0 (checked with the harness) - the scene's clock only updates its `now` when the scene steps, and `RESUME` arrives before the first step. (`this.game.loop.time` would work.) A good puzzle to set the class.

4. Settings

Project: `solutions/ch04_challenge_4_settings_overlay` (from `ch04_pause_and_hud`)

Goal: `sleep` and `wake` between two overlay scenes; `setVisible` on another scene; a setting in the registry that outlives every scene.

`src/scenes/PauseScene.ts`

```
keyboard.on("keydown-S", () => {
  this.scene.launch(SETTINGS_SCENE);
  this.scene.sleep();
});
```

`src/scenes/SettingsScene.ts`

```
keyboard.on("keydown-H", () => {
  const show = this.registry.get(SHOW_HUD) === false;    // it was hidden:
  show it
  this.registry.set(SHOW_HUD, show);
  this.scene.setVisible(show, HUD_SCENE);                // takes effect
  straight away
  this.showSettings();
});
```

```
// wake the pause menu, and take this scene away
private back(): void {
    this.scene.wake(PAUSE_SCENE);
    this.scene.stop();
}
```

src/scenes/HudScene.ts

```
this.scene.setVisible(this.registry.get(SHOW_HUD) !== false);
```

- the pause menu **sleeps**, so it is neither drawn nor listening to keys while the settings are open - and when it wakes, it is exactly as it was (its `once` listeners for P, R and Q are still waiting). Its S listener is `on`, so the settings can be opened again
- the HUD is hidden with `setVisible(false)`, not `slept`: it keeps running and keeps listening, so its numbers are right when it is shown again
- the setting lives in the registry; the HUD reads it in `create()`, so it survives the HUD being stopped and relaunched on a restart

Tested with the harness: S from the pause menu - `PauseScene` sleeping, `SettingsScene` running; H - HUD hidden; ESC - pause menu awake; R - new round, HUD still hidden; score changed while hidden, then H again - HUD visible showing the new score.

Look for: the pause menu slept, not stopped (stopping it would lose its data - the score, the title); the setting surviving a restart. `this.scene.sleep(HUD_SCENE)` for the HUD also works, but a student should be able to say what is different (a sleeping HUD would still receive the events - its listeners are still on `GameScene` - but would not be updated or drawn).

5. Fade in, fade out

Project: `solutions/ch04_challenge_5_fade_in_fade_out` (from `ch04_pause_and_hud`)

Goal: camera fades and their completion event; and the subtle part - a paused scene's camera does not run its effects.

src/scenes/MenuScene.ts

```

this.input.keyboard!.once("keydown-SPACE", () => {
  this.cameras.main.fadeOut(FADE_TIME);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, ()
=> {
    this.scene.start(GAME_SCENE);
  });
});

```

src/scenes/PauseScene.ts

```

// CHALLENGE 5: fade to black, then do `next`.
//
// It is THIS scene's camera that fades. GameScene is paused, and a paused
scene's cameras are
// not updated either - a fade started on its camera would never finish. A
camera's fade covers
// everything under it, so fading the top scene fades the whole screen.
private fadeOutThen(next: () => void): void {
  if (this.leaving) {
    return;
  }
  this.leaving = true;
  this.cameras.main.fadeOut(FADE_TIME);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE,
next);
}

```

plus `this.cameras.main.fadeIn(FADE_TIME)` at the start of `create()` in `MenuScene`, `GameScene` and `HudScene` (the HUD has its own camera, so it needs its own fade).

Tested with the harness: a fade started on the paused `GameScene`'s camera stayed at `fadeEffect.progress 0` for a full second; fading `PauseScene`'s camera darkened the whole screen, game and HUD included, and the restart happened when it finished. Pressing P and R during a Q fade did nothing.

Look for: the scene change inside the `FADE_OUT_COMPLETE` listener, not straight after `fadeOut()` (then there is no fade at all); protection against a second key during the fade (`once` on the menu; the `leaving` flag in the pause menu, which also stops P "carrying on" into a scene that is about to restart). Students who try to fade `GameScene`'s camera from the pause menu will find it never finishes - excellent, if they can explain why.

6. Auto-pause, without a leak

Project: `solutions/ch04_challenge_6_auto_pause` (from `ch04_pause_and_hud`)

Goal: listeners on the game's own emitter, and cleaning them up - proved, not assumed.

`src/scenes/GameScene.ts`

```
this.game.events.on(Phaser.Core.Events.BLUR, this.autoPause, this);
this.game.events.on(Phaser.Core.Events.HIDDEN, this.autoPause, this);
```

```
this.game.events.off(Phaser.Core.Events.BLUR, this.autoPause, this);
this.game.events.off(Phaser.Core.Events.HIDDEN, this.autoPause, this);
```

(in the `SHUTDOWN` handler, next to stopping the HUD)

```
private autoPause(): void {
    if (this.scene.isActive()) {
        this.pauseGame();
    }
}
```

`src/scenes/HudScene.ts`

```
override update(): void {
    const blur = this.game.events.listenerCount(Phaser.Core.Events.BLUR);
    const hidden = this.game.events.listenerCount(Phaser.Core.Events.HIDDEN);
    this.listenersText.setText(`game listeners - BLUR: ${blur}  HIDDEN:
    ${hidden}`);
}
```

`isActive()` is `false` while the scene is paused, so the pause menu is never opened twice, and nothing happens on the game-over screen.

The counts are not 1: Phaser itself listens on the game's emitter (and some of its listeners belong to whichever scenes are running), so the number to check is that it is **the same** in the same situation, round after round. Tested with the harness: during play HIDDEN stayed at 2 and BLUR at 4 through three restarts (BLUR is 5 in the very first round - one of Phaser's

own listeners goes away after the first start, which is why "does not grow" is the test, not "stays exactly the same"). In a copy with the `off` lines removed, three restarts took HIDDEN from 2 to 5 and BLUR from 5 to 7. `game.events.emit("blur")` (from the harness) opened the pause menu during play, did nothing when already paused, and did nothing on the menu after quitting.

Look for: the `off` calls with the same method and context as the `on` calls (an arrow function passed to both `on` and `off` removes nothing); the "only while playing" check; a test plan. `once` is a plausible wrong answer: it would pause the first time only.

Slides

Chapter 4

The life of a scene

What Phaser calls, and when - and the scene manager's other powers



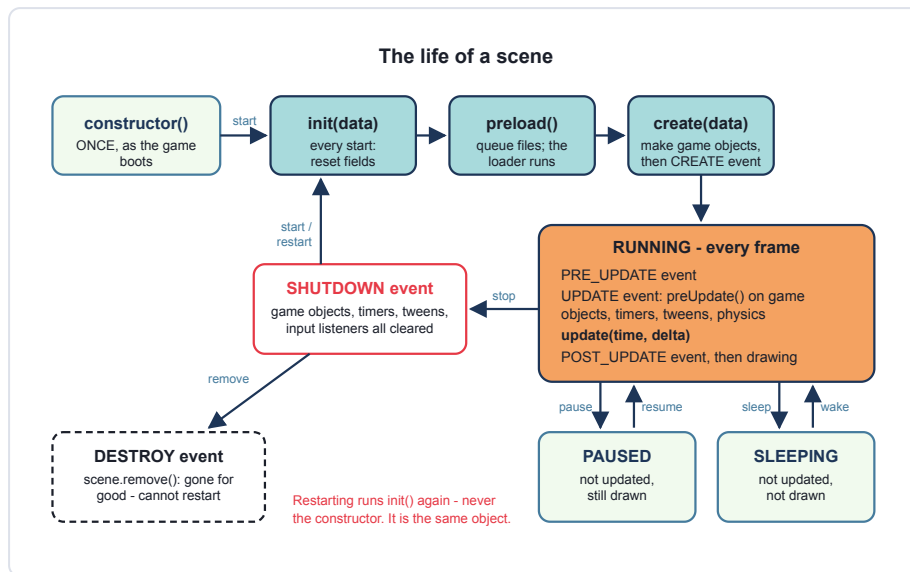
1 / 20

Today

- the order: `constructor` , `init` , `preload` , `create` , `update`
- one frame, step by step; scene **events**
- a `Sprite` 's own `preUpdate` - and `super`
- `launch` , `pause / resume` , `sleep / wake` , `stop` , `remove`
- cleaning up: the listeners Phaser does **not** remove
- `time` vs `delta` ; the frame rate
- a HUD and a pause menu, as scenes

2 / 20

The life of a scene



3 / 20

Once, or every start?

```

constructor() {
    super(DEMO_SCENE);
    eventLog.add("constructor()");
}

// Runs every time the scene starts: the place to reset fields
init(): void {
    this.log("init()");
    this.updates = 0;
    ...
}
    
```

- constructor: **once**, when the game boots
- `init()`, `preload()`, `create()` : **every** start - then the `CREATE` event
- restart: `init()` again, never the constructor. Same object

4 / 20

Try it: Lifecycle Logger

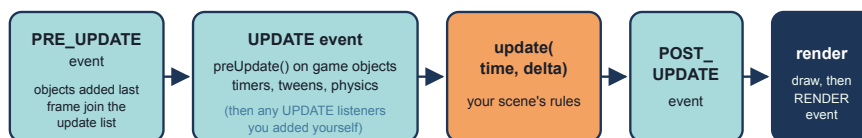
- build, serve, open `ch04_lifecycle_logger`
- read the log from the top: what happens in frame 0? frame 2?
- press **5** (restart): what is missing from the log this time?
- press **S**: what does `super.preUpdate()` do?



5 / 20

One frame, in order

One frame, for one running scene



With several scenes running

every scene is UPDATED first - the last scene in the list first, the first scene last - then every scene is DRAWN - the first scene in the list first, so later scenes are drawn on top.

A paused scene is skipped in the update pass but still drawn. A sleeping scene is skipped in both.

`update()` comes after every game object has moved itself.

6 / 20

Listening for scene events

```
this.events.on(Phaser.Scenes.Events.PAUSE, this.onPause, this);
this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);
...
this.game.events.on(Phaser.Core.Events.STEP, this.onGameStep, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, this.onShutdown, this);
```

- `on(event, method, context)` - the **context** is the object to call it on
- Java: `this::onPause` remembers its object. JavaScript's `this.onPause` does not
- the same three arguments take it off again: `off(event, method, context)`

7 / 20

A Sprite's own `preUpdate`

Coin.ts

```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);
    ...
    this.age = this.age + delta;
    ...
}
```

- `Image` has no `preUpdate` - `Sprite` does: it moves the animation on
- `override` and `protected` - the compiler checks
- `super.preUpdate(...)` - **nobody** checks. Leave it out: moves, never animates

8 / 20

The scene manager

	Updated?	Drawn?	Game objects
<code>launch(key)</code> - alongside this one	yes	yes	new
<code>pause(key)</code> / <code>resume(key)</code>	no	yes	kept
<code>sleep(key)</code> / <code>wake(key)</code>	no	no	kept
<code>stop(key)</code>	no	no	destroyed
<code>remove(key)</code>	-	-	gone for good

`start` stops **the caller**. `restart()` has no key. Requests are **queued**: they happen as the scene manager's next update begins.

9 / 20

Paused or asleep?

- **paused**: frozen, still on screen
- **asleep**: not drawn either
- both keep everything
- neither hears the keyboard - so the keys live in **another** scene

```

- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2 Spinner.preUpdate()
2 UPDATE event
2 update()
2 POST UPDATE event
2 RENDER event
112 >> scene.pause(DEMO_SCENE)
112 PAUSE event
148 >> scene.resume(DEMO_SCENE)
148 RESUME event
148 PRE UPDATE event
148 Spinner.preUpdate()
148 UPDATE event
148 update()
148 POST UPDATE event
148 RENDER event
174 >> scene.sleep(DEMO_SCENE)
174 SLEEP event

DemoScene: SLEEPING isActive: no isPaused: no isSleeping: yes isVisible: no
fps: 53.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

```

10 / 20

Cleaning up

When a scene shuts down, Phaser clears:

- its game objects, timers, tweens, input listeners

It does **not** clear listeners on:

- the scene's own `this.events` (the object - and emitter - is reused)
- `this.game.events`, `this.registry.events`, another scene's `events`

Whatever you add to an emitter that outlives your scene, take off in `SHUTDOWN`.

11 / 20

A leak you can see

Leak mode (L), then restart twice (5, 5):

- every event logged **three** times
- "game steps seen" climbs 3x too fast
- 3 listeners for `CREATE`, 3 for `STEP`

In a game: doubled scores, tripled sounds... then a crash.

DemoScene

update() calls: 68
game steps seen: 201
time: 7776 ms
this.time.now: 7776 ms
delta: 20.0 ms



```
244 POST_UPDATE event
244 POST_UPDATE event
244 RENDER event
244 >> demo.scene.restart()
263 SHUTDOWN event
263 leak mode: listeners left behind!
263 init()
263 preload()
263 loader: complete
263 create()
263 CREATE event
263 CREATE event
263 CREATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 UPDATE event
263 Spinner.preUpdate()
263 UPDATE event
263 update()
263 POST_UPDATE event
263 POST_UPDATE event
263 POST_UPDATE event
263 RENDER event
```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 45.8 listeners for CREATE: 3 for the game's STEP: 3 leak mode: ON super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

12 / 20

Time and delta

- `time` / `this.time.now` - the **game's** clock. Keeps going while a scene is paused
- `delta` - ms since the last frame. Only arrives while the scene **runs**
- `timers` (`this.time.delayedCall`) belong to the scene: they wait during a pause

```
fps: {  
  target: 60,  
  limit: 0,  
},
```

`limit: 10` - the coin moves just as fast, in bigger steps

13 / 20

Try it: Pause and HUD

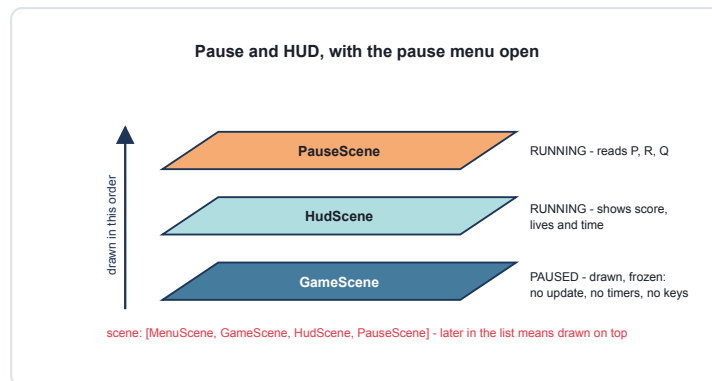
```
scene: [MenuScene, GameScene, HudScene,  
PauseScene],
```

- arrows, collect coins, avoid triangles
- P pauses; R, Q from the menu
- which scenes are running when paused?



14 / 20

Scenes side by side



```
const hudData: HudData = { score: this.score, lives: this.lives, seconds: this.secondsShown };
this.scene.launch(HUD_SCENE, hudData);
```

15 / 20

The HUD listens

```
this.events.emit(SCORE_CHANGED, this.score);
```

```
const gameScene = this.scene.get(GAME_SCENE);
gameScene.events.on(SCORE_CHANGED, this.showScore, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  gameScene.events.off(SCORE_CHANGED, this.showScore, this);
  ...
});
```

GameScene emits; HudScene listens. GameScene 's emitter outlives the HUD - so the HUD cleans up.

16 / 20

Pausing

```
private pauseGame(): void {  
    const data: PauseData = { gameOver: false, title: "Paused",  
score: this.score };  
    this.scene.launch(PAUSE_SCENE, data);  
    this.scene.pause();  
}
```

```
private carryOn(): void {  
    this.scene.resume(GAME_SCENE);  
    this.scene.stop();  
}
```

GameScene pauses itself; PauseScene (running) resumes it



17 / 20

Time that survives a pause

```
this.player.setAlpha(0.4);  
this.time.delayedCall(SAFE_TIME, () => {  
    this.safe = false;  
    this.player.setAlpha(1);  
});
```

Not `this.safeUntil = this.time.now + SAFE_TIME - this.time.now` keeps going during a pause.
Measure time with **timers** and `delta` (as `Coin` ages itself).

18 / 20

Summary

- constructor **once**; `init` , `preload` , `create` every start; `update` every frame
- frame: `PRE_UPDATE` , `UPDATE` (objects' `preUpdate`), `update()` , `POST_UPDATE` , `draw`
- `Sprite` subclass: `protected` override `preUpdate` + `super.preUpdate(time, delta)`
- `launch` alongside; `pause` = frozen but drawn; `sleep` = hidden too; `stop` ; `remove`
- scenes are drawn in list order; `bringToTop`
- clean up listeners on long-lived emitters in `SHUTDOWN`
- pausable time: timers and `delta`

19 / 20

Challenges

1. **Count the frames drawn** - `RENDER` events vs `update()` calls
2. **Thirty frames a second** - `fps.limit` , and a real frame counter on the HUD
3. **How long was I away?** - time paused, counted once
4. **Settings** - a settings scene; the pause menu sleeps; H hides the HUD
5. **Fade in, fade out** - camera fades between scenes
6. **Auto-pause, without a leak** - `BLUR` / `HIDDEN` , proved leak-free

Next: **Chapter 5 - Audio**

20 / 20